

**MINISTRY OF INFORMATION AND COMMUNICATIONS
POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY**



**SOFTWARE ARCHITECTURE AND DESIGN
TIÊU LUẬN**

Lecturer	PGS. TS. Trần Đình Quế
Class	E22CNPM01
Group	01
Student	Đỗ Thùy Anh - B22DCCN015

Hanoi, 2026

Mục lục

1 Từ Monolithic đến Microservices và DDD	6
1.1 Giới thiệu Monolithic Architecture	6
1.1.1 Khái niệm	6
1.1.2 Cấu trúc điển hình	6
1.1.3 Ví dụ thực tế	6
1.1.4 Nhược điểm chi tiết	6
1.1.5 Khi nào nên dùng Monolithic	7
1.2 Microservices Architecture	7
1.2.1 Khái niệm	7
1.2.2 Đặc điểm	7
1.2.3 So sánh Monolithic vs Microservices	7
1.2.4 Ưu điểm	7
1.2.5 Nhược điểm	7
1.2.6 Nguyên tắc thiết kế	8
1.3 Domain Driven Design (DDD)	8
1.3.1 Mục tiêu	8
1.3.2 Các khái niệm cốt lõi	8
1.3.3 Context Map	9
1.3.4 DDD trong Microservices	9
1.4 Case Study: Healthcare (Luyện Decomposition)	9
1.4.1 Mô tả bài toán	9
1.4.2 Bước 1: Xác định Domain	9
1.4.3 Bước 2: Xác định Bounded Context	9
1.4.4 Bước 3: Phân rã thành Microservices	10

1.4.5	Bước 4: Xác định quan hệ	10
1.4.6	Ví dụ API	10
1.5	Kết luận	10
2	Phát triển Hệ E-Commerce Microservices	11
2.1	Xác định yêu cầu	11
2.1.1	Functional Requirements	11
2.1.2	Non-functional Requirements	11
2.2	Phân rã hệ thống theo DDD	11
2.2.1	Bounded Context	11
2.2.2	Nguyên tắc	12
2.3	Thiết kế Product Service (Django)	12
2.3.1	Phân loại sản phẩm	12
2.3.2	Model tổng quát	12
2.3.3	Chi tiết theo domain	12
2.3.4	API	13
2.4	Thiết kế User Service (Django)	13
2.4.1	Phân loại người dùng	13
2.4.2	Model	13
2.4.3	Phân quyền (RBAC)	14
2.4.4	API	14
2.5	Thiết kế Cart Service	14
2.5.1	Model	14
2.5.2	Logic	14
2.5.3	API	14
2.6	Thiết kế Order Service	15
2.6.1	Model	15
2.6.2	Workflow	15
2.7	Thiết kế Payment Service	15
2.7.1	Model	15
2.7.2	Trạng thái	15

2.7.3 API	15
2.8 Thiết kế Shipping Service	16
2.8.1 Model	16
2.8.2 Trạng thái	16
2.8.3 API	16
2.9 Luồng hệ thống tổng thể	16
2.10 Hướng dẫn thực hành	16
2.10.1 Mục tiêu	16
2.10.2 Hướng dẫn vẽ Class Diagram bằng Visual Paradigm	17
2.10.3 Mapping Class Diagram sang Database	18
2.10.4 Thiết kế Database cho từng Service	18
2.10.5 So sánh MySQL vs PostgreSQL	21
2.10.6 Bài tập	21
2.10.7 Checklist đánh giá	21
2.11 Kết luận	21
3 AI Service cho tư vấn sản phẩm	22
3.1 Mục tiêu	22
3.2 Kiến trúc AI Service	22
3.3 Thu thập dữ liệu	22
3.3.1 User Behavior Data	22
3.3.2 Ví dụ dataset	23
3.4 Mô hình LSTM (Sequence Modeling)	23
3.4.1 Ý tưởng	23
3.4.2 Model chi tiết	23
3.4.3 Training	23
3.5 Knowledge Graph với Neo4j	24
3.5.1 Mô hình đồ thị	24
3.5.2 Ví dụ Cypher	24
3.5.3 Truy vấn gợi ý	24
3.6 RAG (Retrieval-Augmented Generation)	24

3.6.1 Pipeline	24
3.6.2 Vector Database	24
3.6.3 Ví dụ	25
3.7 Kết hợp Hybrid Model	25
3.8 Hai dạng AI Service	25
3.8.1 1. Recommendation List	25
3.8.2 2. Chatbot tư vấn	25
3.9 Triển khai AI Service	26
3.9.1 Tech stack	26
3.9.2 Kiến trúc	26
3.10 Bài tập	26
3.11 Checklist đánh giá	26
3.12 Kết luận	27
4 Xây dựng hệ thống hoàn chỉnh	28
4.1 Kiến trúc tổng thể	28
4.1.1 Mô hình hệ thống	28
4.1.2 Kiến trúc hệ thống (System Architecture)	29
4.1.3 Stack công nghệ	31
4.2 Chi tiết từng Service	31
4.2.1 1. API Gateway (Nginx)	31
4.2.2 2. Product Service	31
4.2.3 3. Comment-Rate Service	32
4.2.4 4. Recommender AI Service	33
4.3 Luồng hệ thống End-to-End	34
4.3.1 Use Case 1: Xem sản phẩm và đánh giá	34
4.3.2 Use Case 2: Chatbot AI tư vấn	34
4.3.3 Use Case 3: Mua hàng	35
4.4 Database Schema	35
4.4.1 Product Service Database	35
4.4.2 Comment-Rate Service Database	35

4.5 Docker Deployment	36
4.5.1 Cấu trúc docker-compose.yml	36
4.5.2 Ví dụ cấu hình thực tế	36
4.5.3 Dockerfile (Django service)	37
4.5.4 Cấu hình Nginx Gateway thực tế	37
4.5.5 Khởi chạy hệ thống	37
4.6 Authentication & Security	38
4.6.1 JWT Authentication	38
4.6.2 RBAC (Role-Based Access Control)	38
4.6.3 Membership & Discount System	39
4.7 Monitoring Logging	39
4.7.1 Health Checks	39
4.7.2 Logs	39
4.8 Performance Considerations	39
4.8.1 Scalability	39
4.8.2 Caching	39
4.8.3 Database Indexing	40
4.9 Troubleshooting	40
4.9.1 Reviews không hiển thị	40
4.9.2 AI Chat lỗi kết nối	40
4.9.3 Hình ảnh sản phẩm không hiển thị	40
4.10 Bài tập thực hành sinh viên	40
4.10.1 Mục tiêu	40
4.10.2 Step-by-step	41
4.10.3 Verification Checklist	41
4.11 Kết luận	41
Kết luận	42

Chương 1 Từ Monolithic đến Microservices và DDD

1.1 Giới thiệu Monolithic Architecture

1.1.1 Khái niệm

Monolithic Architecture là mô hình trong đó toàn bộ hệ thống (UI, business logic, database access) được xây dựng và triển khai như một khối duy nhất.

1.1.2 Cấu trúc điển hình

- Presentation Layer (UI)
- Business Logic Layer
- Data Access Layer

1.1.3 Ví dụ thực tế

Một hệ e-commerce monolithic sẽ bao gồm:

- Quản lý sản phẩm
- Giỏ hàng
- Thanh toán
- User

tất cả nằm trong cùng một codebase.

1.1.4 Nhược điểm chi tiết

- **Khó mở rộng:** scale toàn hệ thống thay vì từng module
- **Coupling cao:** thay đổi nhỏ ảnh hưởng toàn bộ hệ thống
- **Deploy rủi ro:** lỗi nhỏ có thể làm sập toàn hệ
- **Khó phát triển nhóm:** nhiều team cùng sửa một codebase

1.1.5 Khi nào nên dùng Monolithic

- Hệ thống nhỏ
- MVP (Minimum Viable Product)
- Team ít người

1.2 Microservices Architecture

1.2.1 Khái niệm

Microservices là kiến trúc chia hệ thống thành các dịch vụ nhỏ, độc lập, mỗi service thực hiện một chức năng riêng biệt.

1.2.2 Đặc điểm

- Mỗi service có database riêng
- Giao tiếp qua API (REST/gRPC)
- Deploy độc lập

1.2.3 So sánh Monolithic vs Microservices

Tiêu chí	Monolithic	Microservices
Deploy	Một lần	Nhiều lần
Scale	Toàn hệ	Từng service
Coupling	Cao	Thấp

1.2.4 Ưu điểm

- Scale độc lập từng service
- Tăng tốc phát triển
- Phù hợp hệ thống lớn

1.2.5 Nhược điểm

- Phức tạp hệ thống
- Quản lý distributed system
- Debug khó hơn

1.2.6 Nguyên tắc thiết kế

- Single Responsibility
- Loose Coupling
- High Cohesion

1.3 Domain Driven Design (DDD)

1.3.1 Mục tiêu

DDD giúp mô hình hóa hệ thống theo nghiệp vụ (business domain), không phụ thuộc công nghệ.

1.3.2 Các khái niệm cốt lõi

Entity

Đối tượng có định danh (ID).

- Ví dụ: User, Product

Value Object

Không có ID, so sánh bằng giá trị.

- Ví dụ: Address, Money

Aggregate

Nhóm các entity liên quan.

- Ví dụ: Order + OrderItem

Bounded Context

Ranh giới domain rõ ràng.

- Ví dụ: User Context, Order Context

1.3.3 Context Map

Sinh viên cần vẽ sơ đồ thể hiện mối quan hệ giữa các bounded context:

- Shared Kernel
- Customer-Supplier

1.3.4 DDD trong Microservices

- Mỗi bounded context = 1 microservice
- Tránh chia service theo technical layer

1.4 Case Study: Healthcare (Luyện Decomposition)

1.4.1 Mô tả bài toán

Hệ thống quản lý bệnh viện gồm:

- Quản lý bệnh nhân
- Quản lý bác sĩ
- Đặt lịch khám

1.4.2 Bước 1: Xác định Domain

- Patient Management
- Doctor Management
- Appointment Scheduling

1.4.3 Bước 2: Xác định Bounded Context

- Patient Context
- Doctor Context
- Appointment Context

1.4.4 Bước 3: Phân rã thành Microservices

- Patient Service
- Doctor Service
- Appointment Service

1.4.5 Bước 4: Xác định quan hệ

- Appointment phụ thuộc Patient và Doctor
- Giao tiếp qua API

1.4.6 Ví dụ API

```
1 GET /patients
2 GET /doctors
3 POST /appointments
```

1.5 Kết luận

- Monolithic phù hợp hệ nhỏ
- Microservices phù hợp hệ lớn
- DDD là nền tảng quan trọng để phân rã hệ thống đúng

Chương 2 Phát triển Hệ E-Commerce Microservices

2.1 Xác định yêu cầu

2.1.1 Functional Requirements

- Quản lý sản phẩm (đa domain: book, electronics, fashion)
- Quản lý người dùng (admin, staff, customer)
- Giỏ hàng (cart)
- Đặt hàng (order)
- Thanh toán (payment)
- Giao hàng (shipping)
- Tìm kiếm và gợi ý sản phẩm

2.1.2 Non-functional Requirements

- **Scalability:** scale từng service độc lập
- **High Availability:** hệ thống luôn sẵn sàng
- **Security:** JWT, authentication
- **Maintainability:** dễ bảo trì

2.2 Phân rã hệ thống theo DDD

2.2.1 Bounded Context

- User Context → user-service
- Product Context → product-service
- Cart Context → cart-service
- Order Context → order-service

- Payment Context → payment-service
- Shipping Context → shipping-service

2.2.2 Nguyên tắc

- Mỗi context = 1 database riêng
- Giao tiếp qua REST API

2.3 Thiết kế Product Service (Django)

2.3.1 Phân loại sản phẩm

- **Book:** giáo trình, tiểu thuyết
- **Electronics:** mobile, laptop, tủ lạnh, điều hòa
- **Fashion:** áo, quần, giày

2.3.2 Model tổng quát

```

1 class Category(models.Model):
2     name = models.CharField(max_length=100)
3
4 class Product(models.Model):
5     name = models.CharField(max_length=255)
6     price = models.FloatField()
7     stock = models.IntegerField()
8     category = models.ForeignKey(Category, on_delete=models.
        CASCADE)
    
```

2.3.3 Chi tiết theo domain

Book

```

1 class Book(models.Model):
2     product = models.OneToOneField(Product, on_delete=models.
        CASCADE)
3     author = models.CharField(max_length=255)
4     publisher = models.CharField(max_length=255)
5     isbn = models.CharField(max_length=20)
    
```

Electronics

```
1 class Electronics(models.Model):
2     product = models.OneToOneField(Product, on_delete=models.
3         CASCADE)
4     brand = models.CharField(max_length=100)
5     warranty = models.IntegerField()
```

Fashion

```
1 class Fashion(models.Model):
2     product = models.OneToOneField(Product, on_delete=models.
3         CASCADE)
4     size = models.CharField(max_length=10)
5     color = models.CharField(max_length=50)
```

2.3.4 API

```
1 GET /products/
2 POST /products/
3 GET /products/{id}
```

2.4 Thiết kế User Service (Django)

2.4.1 Phân loại người dùng

- Admin: toàn quyền hệ thống
- Staff: xử lý đơn hàng, vận hành
- Customer: mua hàng

2.4.2 Model

```
1 class User(AbstractUser):
2     ROLE_CHOICES = (
3         ('admin', 'Admin'),
4         ('staff', 'Staff'),
5         ('customer', 'Customer'),
6     )
7     role = models.CharField(max_length=20, choices=ROLE_CHOICES)
```

2.4.3 Phân quyền (RBAC)

- Admin: CRUD toàn bộ
- Staff: xử lý order, shipping
- Customer: mua hàng, xem sản phẩm

2.4.4 API

```
1 POST /auth/register
2 POST /auth/login
3 GET /users/
```

2.5 Thiết kế Cart Service

2.5.1 Model

```
1 class Cart(models.Model):
2     user_id = models.IntegerField()
3
4 class CartItem(models.Model):
5     cart = models.ForeignKey(Cart, on_delete=models.CASCADE)
6     product_id = models.IntegerField()
7     quantity = models.IntegerField()
```

2.5.2 Logic

- Add product vào cart
- Update số lượng
- Remove item

2.5.3 API

```
1 POST /cart/add
2 GET /cart/
3 DELETE /cart/remove
```

2.6 Thiết kế Order Service

2.6.1 Model

```

1 class Order(models.Model):
2     user_id = models.IntegerField()
3     total_price = models.FloatField()
4     status = models.CharField(max_length=50)
5
6 class OrderItem(models.Model):
7     order = models.ForeignKey(Order, on_delete=models.CASCADE)
8     product_id = models.IntegerField()
9     quantity = models.IntegerField()
    
```

2.6.2 Workflow

- Tạo order từ cart
- Gửi request sang payment-service
- Sau khi thanh toán → shipping

2.7 Thiết kế Payment Service

2.7.1 Model

```

1 class Payment(models.Model):
2     order_id = models.IntegerField()
3     amount = models.FloatField()
4     status = models.CharField(max_length=50)
    
```

2.7.2 Trạng thái

- Pending
- Success
- Failed

2.7.3 API

```

1 POST /payment/pay
2 GET /payment/status
    
```


2.8 Thiết kế Shipping Service

2.8.1 Model

```
1 class Shipment(models.Model):  
2     order_id = models.IntegerField()  
3     address = models.TextField()  
4     status = models.CharField(max_length=50)
```

2.8.2 Trạng thái

- Processing
- Shipping
- Delivered

2.8.3 API

```
1 POST /shipping/create  
2 GET /shipping/status
```

2.9 Luồng hệ thống tổng thể

1. User đăng nhập (user-service)
2. Xem sản phẩm (product-service)
3. Thêm vào giỏ hàng (cart-service)
4. Tạo đơn hàng (order-service)
5. Thanh toán (payment-service)
6. Giao hàng (shipping-service)

2.10 Hướng dẫn thực hành

2.10.1 Mục tiêu

Sinh viên cần:

- Thiết kế Class Diagram bằng Visual Paradigm (VP)

- Xây dựng database cho từng microservice
- Mapping từ Class Diagram $\rightarrow$ Database

2.10.2 Hướng dẫn vẽ Class Diagram bằng Visual Paradigm

Bước 1: Xác định lớp (Classes)

Sinh viên xác định các lớp chính theo từng service:

Product Service:

- Product
- Category
- Book
- Electronics
- Fashion

User Service:

- User
- Role

Order Service:

- Order
- OrderItem

Bước 2: Xác định thuộc tính

Ví dụ lớp Product:

- id: int
- name: string
- price: float
- stock: int

Bước 3: Xác định quan hệ (Relationships)

- Association: Product $\rightarrow$ Category
- Inheritance: Book, Electronics, Fashion kế thừa Product
- Composition: Order chứa OrderItem

Ký hiệu UML:

- 1..* (one-to-many)
- 1..1 (one-to-one)

Yêu cầu bài nộp:

- Export sơ đồ từ VP (PNG/PDF)
- Có đầy đủ class + relationship

2.10.3 Mapping Class Diagram sang Database

Nguyên tắc

- Class $\rightarrow$ Table
- Attribute $\rightarrow$ Column
- Relationship $\rightarrow$ Foreign Key

Ví dụ

```
1 Product(id, name, price)
2 Category(id, name)
3 => Product.category_id (FK)
```

2.10.4 Thiết kế Database cho từng Service

Nguyên tắc Microservices

- Mỗi service có database riêng với data model tương ứng
- Không share database giữa các service

Product Service Database (PostgreSQL)

Lý do chọn PostgreSQL:

- Hỗ trợ tốt JSON
- Phù hợp dữ liệu phức tạp

```
1 CREATE TABLE category (  
2     id SERIAL PRIMARY KEY,  
3     name VARCHAR(100)  
4 );  
5  
6 CREATE TABLE product (  
7     id SERIAL PRIMARY KEY,  
8     name VARCHAR(255),  
9     price FLOAT,  
10    stock INT,  
11    category_id INT REFERENCES category(id)  
12 );  
13  
14 CREATE TABLE book (  
15     product_id INT PRIMARY KEY,  
16     author VARCHAR(255),  
17     isbn VARCHAR(20)  
18 );
```

User Service Database (MySQL)

Lý do chọn MySQL:

- Phổ biến
- Phù hợp authentication

```
1 CREATE TABLE user (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     username VARCHAR(100),  
4     password VARCHAR(255),  
5     role VARCHAR(20)  
6 );
```

Cart Service

```
1 CREATE TABLE cart (  
2     id INT PRIMARY KEY,  
3     user_id INT
```

```
4 );  
5  
6 CREATE TABLE cart_item (  
7     id INT PRIMARY KEY,  
8     cart_id INT,  
9     product_id INT,  
10    quantity INT  
11 );
```

Order Service

```
1 CREATE TABLE orders (  
2     id INT PRIMARY KEY,  
3     user_id INT,  
4     total_price FLOAT,  
5     status VARCHAR(50)  
6 );  
7  
8 CREATE TABLE order_item (  
9     id INT PRIMARY KEY,  
10    order_id INT,  
11    product_id INT,  
12    quantity INT  
13 );
```

Payment Service

```
1 CREATE TABLE payment (  
2     id INT PRIMARY KEY,  
3     order_id INT,  
4     amount FLOAT,  
5     status VARCHAR(50)  
6 );
```

Shipping Service

```
1 CREATE TABLE shipment (  
2     id INT PRIMARY KEY,  
3     order_id INT,  
4     address TEXT,  
5     status VARCHAR(50)  
6 );
```

2.10.5 So sánh MySQL vs PostgreSQL

Tiêu chí	MySQL	PostgreSQL
Hiệu năng	Tốt	Tốt
JSON	Trung bình	Mạnh
Quan hệ phức tạp	Trung bình	Tốt

2.10.6 Bài tập

- Vẽ Class Diagram cho toàn bộ hệ thống bằng VP
- Mapping sang database schema
- Triển khai database bằng MySQL/PostgreSQL

2.10.7 Checklist đánh giá

- Có sơ đồ class đúng UML
- Có mapping rõ ràng sang database
- Database tách riêng từng service
- Có sử dụng cả MySQL và PostgreSQL

2.11 Kết luận

- Kiến trúc microservices giúp hệ thống linh hoạt
- Django phù hợp xây dựng nhanh
- DDD giúp thiết kế đúng ngay từ đầu

Chương 3 AI Service cho tư vấn sản phẩm

3.1 Mục tiêu

Xây dựng hệ thống AI gợi ý sản phẩm dựa trên:

- Hành vi người dùng (click, search, add-to-cart)
- Quan hệ sản phẩm (similarity)
- Ngữ cảnh truy vấn (chatbot)

Output:

- Danh sách sản phẩm đề xuất
- Chatbot tư vấn

3.2 Kiến trúc AI Service

AI Service được thiết kế như một microservice độc lập:

- Input: user behavior, query
- Processing:
 - LSTM model
 - Knowledge Graph
 - RAG
- Output: recommendation / chatbot response

3.3 Thu thập dữ liệu

3.3.1 User Behavior Data

- user_id

- product_id
- action (view, click, add_to_cart)
- timestamp

3.3.2 Ví dụ dataset

```

1 user_id, product_id, action, time
2 1, 101, view, t1
3 1, 102, add_to_cart, t2

```

3.4 Mô hình LSTM (Sequence Modeling)

3.4.1 Ý tưởng

Dự đoán sản phẩm tiếp theo dựa trên chuỗi hành vi.

3.4.2 Model chi tiết

```

1 import torch
2 import torch.nn as nn
3
4 class LSTMModel(nn.Module):
5     def __init__(self, input_dim=10, hidden_dim=64, output_dim
6         =100):
7         super().__init__()
8         self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=
9             True)
10        self.fc = nn.Linear(hidden_dim, output_dim)
11
12    def forward(self, x):
13        out, _ = self.lstm(x)
14        out = out[:, -1, :]
15        return self.fc(out)

```

3.4.3 Training

```

1 criterion = nn.CrossEntropyLoss()
2 optimizer = torch.optim.Adam(model.parameters())
3
4 for epoch in range(epochs):
5     output = model(x)
6     loss = criterion(output, y)
7     loss.backward()

```



```
8 optimizer.step()
```

3.5 Knowledge Graph với Neo4j

3.5.1 Mô hình đồ thị

- Node: User, Product
- Edge:
 - BUY
 - VIEW
 - SIMILAR

3.5.2 Ví dụ Cypher

```
1 CREATE (u:User {id:1})
2 CREATE (p:Product {id:101})
3 CREATE (u)-[:BUY]->(p)
```

3.5.3 Truy vấn gợi ý

```
1 MATCH (u:User {id:1})-[:BUY]->(p)-[:SIMILAR]->(rec)
2 RETURN rec
```

3.6 RAG (Retrieval-Augmented Generation)

3.6.1 Pipeline

- Retrieve:
 - Tìm sản phẩm liên quan từ DB / vector DB
- Generate:
 - Sinh câu trả lời bằng LLM

3.6.2 Vector Database

- FAISS / ChromaDB
- Embedding từ mô tả sản phẩm

3.6.3 Ví dụ

```
1 query = "laptop gaming"
2 results = vector_db.search(query)
3 response = LLM.generate(results)
```

3.7 Kết hợp Hybrid Model

- LSTM: dự đoán hành vi
- Graph: quan hệ sản phẩm
- RAG: hiểu ngữ nghĩa

Final Recommendation:

```
1 final_score = w1 * lstm + w2 * graph + w3 * rag
```

3.8 Hai dạng AI Service

3.8.1 1. Recommendation List

Use cases:

- Khi search
- Khi add-to-cart

API:

```
1 GET /recommend?user_id=1
```

Output:

```
1 [101, 102, 205]
```

3.8.2 2. Chatbot tư vấn

Input:

```
1 "t i c n laptop gi r "
```

Pipeline:

- NLP hiểu intent

- Retrieve sản phẩm
- Generate response

API:

1 POST /chatbot

Output:

1 " B n c t h tham k h o Laptop XYZ gi 10 triu ..."

3.9 Triển khai AI Service

3.9.1 Tech stack

- tensorflow/PyTorch (LSTM)
- Neo4j (Graph)
- FAISS (Vector DB)
- FastAPI (service)

3.9.2 Kiến trúc

- AI service độc lập
- Giao tiếp với các service khác qua API

3.10 Bài tập

- Xây dựng model LSTM đơn giản
- Tạo graph trong Neo4j
- Implement API recommendation
- Xây dựng chatbot cơ bản

3.11 Checklist đánh giá

- Có pipeline AI rõ ràng
- Có model (LSTM)

- Có Graph và RAG
- Có API hoạt động

3.12 Kết luận

- AI giúp cá nhân hóa trải nghiệm
- Kết hợp nhiều mô hình cho hiệu quả cao
- Phù hợp hệ e-commerce hiện đại

Chương 4 Xây dựng hệ thống hoàn chỉnh

4.1 Kiến trúc tổng thể

4.1.1 Mô hình hệ thống

Hệ thống **ecom-final** được xây dựng theo kiến trúc microservices phân tán hoàn toàn. Mỗi service là một Django project độc lập, chạy trong container riêng biệt. Tổng cộng gồm 13 container (11 services + 1 PostgreSQL + Neo4j):

- **frontend / api-gateway** (Django + Nginx, port 80/8000) — Điểm vào duy nhất. Django xử lý session và render UI, Nginx đóng vai trò reverse proxy định tuyến đến từng service.
- **user-service** (customer-service, port 8010) — Quản lý khách hàng, thông tin cá nhân và hạng thành viên (Bronze/Silver/Gold/Platinum).
- **auth-service** (port 8011) — Xác thực JWT độc lập, tách biệt với user-service theo nguyên tắc Single Responsibility.
- **product-service** (port 8001, PostgreSQL) — Quản lý sản phẩm với 10 danh mục điện tử; dùng PostgreSQL vì hỗ trợ JSON tốt cho attributes đa dạng.
- **cart-service** (port 8003, MySQL) — Quản lý giỏ hàng, tự động tạo cart khi khách hàng đăng ký.
- **order-service** (port 8004, MySQL) — Tạo và quản lý đơn hàng; gọi payment-service và shipping-service theo chuỗi (Saga pattern đơn giản).
- **payment-service** (pay-service, port 8005, MySQL) — Xử lý thanh toán, trả về trạng thái Pending/Paid/Failed.
- **shipping-service** (ship-service, port 8006, MySQL) — Quản lý vận chuyển, cập nhật trạng thái giao hàng.
- **comment-rate-service** (port 8007, MySQL) — Đánh giá sản phẩm 1–5 sao; chỉ cho phép khách hàng đã mua hàng đánh giá.
- **staff-service** (port 8008, MySQL) — Quản lý nhân viên vận hành.
- **manager-service** (port 8009, MySQL) — Quản lý toàn bộ hệ thống từ phía quản lý.

- **recommender-ai-service** (port 8888, Neo4j) — AI gợi ý sản phẩm và chatbot tư vấn bằng RAG + LLM.

4.1.2 Kiến trúc hệ thống (System Architecture)

Hệ thống tuân thủ các nguyên tắc thiết kế enterprise hiện đại:

Microservice Architecture

Mỗi business domain được tách thành một service độc lập theo nguyên tắc DDD:

- **User Service:** Xử lý xác thực, phân quyền và quản lý người dùng.
- **Product Service:** Quản lý danh mục sản phẩm và tồn kho.
- **Order Service:** Xử lý vòng đời đơn hàng (tạo → thanh toán → giao hàng).
- **Payment Service:** Xử lý giao dịch thanh toán.
- **AI Service:** Gợi ý sản phẩm và chatbot tư vấn thông minh.

Mỗi service duy trì database riêng (database-per-service pattern), đảm bảo không có sự phụ thuộc dữ liệu trực tiếp.

API Gateway

API Gateway (Nginx) là điểm vào duy nhất (single entry point), chịu trách nhiệm:

- Định tuyến request tới microservice phù hợp.
- Xác thực JWT token trước khi chuyển tiếp.
- Áp dụng rate limiting và chính sách bảo mật.
- Ghi nhật ký và monitoring API.

Service Communication

Hệ thống áp dụng chiến lược giao tiếp kép (hybrid):

Giao tiếp đồng bộ (Synchronous) — REST API qua HTTP:

```
1 # V du: order-service gọi sang payment-service
2 import requests
3 response = requests.post(
4     "http://payment-service:8000/payments/create/",
5     json={"order_id": order.id, "amount": total},
6     timeout=10
7 )
```

Giao tiếp bất đồng bộ (Asynchronous) — Message queue (Redis/RabbitMQ): Khi đơn hàng được tạo, một event được publish và consumed bởi payment-service và notification-service mà không cần chờ phản hồi trực tiếp. Điều này giúp hệ thống không bị block khi một service bị chậm.

Cấu trúc thư mục dự án

```
1 ecom-final/  
2 |-- api-gateway/           # Django + Nginx (UI + routing)  
3 |-- customer-service/     # User management service  
4 |-- product-service/      # Product catalog (PostgreSQL)  
5 |-- cart-service/         # Shopping cart  
6 |-- order-service/        # Order lifecycle  
7 |-- pay-service/          # Payment processing  
8 |-- ship-service/         # Shipping management  
9 |-- comment-rate-service/ # Reviews & ratings  
10 |-- recommender-ai-service/ # AI + Neo4j  
11 |-- auth-service/        # JWT authentication  
12 |-- docker-compose.yml    # Orchestration  
13 |-- nginx.conf            # Gateway routing  
14 |-- setup_db.bat          # Data initialization
```

Nguyên tắc thiết kế (Design Principles)

- **Loose Coupling:** Các service chỉ giao tiếp qua API, không dùng chung DB.
- **High Cohesion:** Mỗi service đóng gói duy nhất một business domain.
- **Scalability:** Scale từng service độc lập theo tải thực tế.
- **Fault Isolation:** Lỗi ở một service không làm sập toàn hệ thống.

Security

Bảo mật được thực thi qua nhiều lớp:

- JWT-based authentication (auth-service độc lập).
- API Gateway validation cho mọi request.
- Role-Based Access Control: Admin (toàn quyền), Staff (xử lý đơn), Customer (mua hàng).
- CSRF protection trên tất cả POST endpoints của Django.

4.1.3 Stack công nghệ

Thành phần	Công nghệ	Lý do chọn
Backend	Django 5.2 + DRF 3.14	Rapid development, ORM mạnh
DB quan hệ	MySQL 8.0	Phổ biến, ACID-compliant
DB sản phẩm	PostgreSQL 15	JSON support, quan hệ phức tạp
Graph DB	Neo4j 5.10	Knowledge Graph cho AI
Container	Docker + Compose	Nhất quán môi trường
Proxy	Nginx	High-performance reverse proxy
AI/ML	Google Gemini + Transformers	LLM cho chatbot RAG

Bảng 4.1: Stack công nghệ của hệ thống ecom-final

4.2 Chi tiết từng Service

4.2.1 1. API Gateway (Nginx)

Chức năng

- Định tuyến request tới các service phù hợp
- Xử lý authentication/authorization
- Load balancing cơ bản

Cấu hình

```
1 location /users/ {
2     proxy_pass http://user-service:8000;
3 }
4 location /products/ {
5     proxy_pass http://product-service:8001;
6 }
```

4.2.2 2. Product Service

Danh mục sản phẩm (10 loại)

1. Laptops
2. Smartphones
3. Tablets
4. Headphones

5. Smartwatches
6. Keyboards Mice
7. Monitors
8. Cameras
9. Smart Home
10. Storage Backup

Mô hình dữ liệu

```
1 class Product(models.Model):  
2     name = models.CharField(max_length=255)  
3     description = models.TextField(blank=True)  
4     price = models.DecimalField(max_digits=10, decimal_places=2)  
5     stock = models.IntegerField(default=0)  
6     category = models.ForeignKey(Category)  
7     image_url = models.URLField(max_length=500)  
8     created_at = models.DateTimeField(auto_now_add=True)
```

Sample Data

- Tổng 30 sản phẩm được load vào hệ thống
- Mỗi sản phẩm có hình ảnh từ Unsplash CDN
- Giá từ 1.8 triệu (chuột gaming) đến 52 triệu (máy ảnh Nikon)
- Stock từ 3 đến 100 units

4.2.3 3. Comment-Rate Service

Chức năng

- Quản lý đánh giá/bình luận sản phẩm
- Tính toán rating trung bình
- API endpoints cho review CRUD

Mô hình

```

1 class Review(models.Model):
2     product_id = models.IntegerField()
3     customer_id = models.IntegerField()
4     rating = models.IntegerField() # 1-5
5     comment = models.TextField()
6     created_at = models.DateTimeField(auto_now_add=True)

```

API Endpoints

```

1 GET /reviews/?product_id=1
2 POST /reviews/
3 GET /reviews/summary/?product_ids=1,2,3

```

Sample Reviews

- 14 đánh giá mẫu được tạo trong setup
- Từ 4-5 sao từ các khách hàng demo
- Bình luận tiếng Việt chi tiết
- Bao gồm đánh giá cho laptops, smartphones, tablets, headphones, smartwatches, SSDs

4.2.4 4. Recommender AI Service

Mục tiêu

- Gợi ý sản phẩm dựa trên hành vi người dùng
- Chatbot tư vấn mua hàng
- Kết nối giữa frontend và backend AI

Các API Endpoints

```

1 GET /health/ # Health check
2 POST /chatbot/ # Chatbot RAG
3 POST /api/chat/ # AI Chat endpoint (frontend)
4 GET /recommendations/ # L y g i
5 POST /lstm-predict/ # D on LSTM

```

CORS Configuration

```

1 CORS_ALLOWED_ORIGINS = [
2     "http://localhost",
3     "http://127.0.0.1",
4     "http://localhost:80",
5     "http://localhost:8000",
6     "http://localhost:8888",
7     "http://api-gateway:80",
8 ]

```

Frontend Integration

Frontend gửi request AI chat qua JavaScript:

```

1 const response = await fetch(
2     'http://localhost:8888/api/ai/chat/',
3     {
4         method: 'POST',
5         headers: { 'Content-Type': 'application/json' },
6         body: JSON.stringify({
7             user_id: 1,
8             message: userMessage,
9             conversation_id: sessionId
10        })
11    }
12 );

```

4.3 Luồng hệ thống End-to-End

4.3.1 Use Case 1: Xem sản phẩm và đánh giá

1. User truy cập homepage
2. API Gateway routing tới product-service
3. Product service lấy 30 sản phẩm + hình ảnh
4. API Gateway gọi comment-rate-service lấy rating
5. Frontend hiển thị sản phẩm với sao đánh giá

4.3.2 Use Case 2: Chatbot AI tư vấn

1. User click robot icon, nhập câu hỏi
2. JavaScript gửi POST tới /api/ai/chat/

3. AI service xử lý (RAG + LLM)
4. Trả về gợi ý sản phẩm bằng tiếng Việt
5. Frontend hiển thị trong chat bubble

4.3.3 Use Case 3: Mua hàng

1. User add product to cart (cart-service)
2. Checkout tạo order (order-service)
3. Order gọi payment-service
4. Payment success → shipping-service
5. User xem order history

4.4 Database Schema

4.4.1 Product Service Database

```

1 CREATE TABLE category (
2     id INT PRIMARY KEY,
3     name VARCHAR(100) UNIQUE,
4     description TEXT
5 );
6
7 CREATE TABLE product (
8     id INT PRIMARY KEY,
9     name VARCHAR(255),
10    description TEXT,
11    price DECIMAL(10,2),
12    stock INT,
13    category_id INT REFERENCES category(id),
14    image_url VARCHAR(500),
15    created_at TIMESTAMP
16 );
    
```

4.4.2 Comment-Rate Service Database

```

1 CREATE TABLE review (
2     id INT PRIMARY KEY,
3     product_id INT,
4     customer_id INT,
5     rating INT CHECK (rating >= 1 AND rating <= 5),
6     comment TEXT,
7     created_at TIMESTAMP
    
```

8);

4.5 Docker Deployment

4.5.1 Cấu trúc docker-compose.yml

Hệ thống sử dụng Docker Compose để khởi chạy đồng thời tất cả container. Mỗi service được cấu hình với:

- Container riêng biệt, build từ Dockerfile cục bộ.
- Database riêng: MySQL cho cart/order/payment/shipping/comment/staff/manager/auth; PostgreSQL cho product; Neo4j cho AI.
- Health check để đảm bảo DB sẵn sàng trước khi service khởi động (`condition: service_healthy`).
- Network chung `microservices-network` (bridge driver) cho inter-service communication.
- Environment variables cấu hình DB, secret key, URL service khác.

4.5.2 Ví dụ cấu hình thực tế

```

1 # Trích docker-compose.yml - order-service
2 order-service:
3   build: ./order-service
4   ports:
5     - "8004:8000"
6   environment:
7     DB_ENGINE: mysql
8     DB_NAME: order_db
9     DB_HOST: mysql-order
10    PAYMENT_SERVICE_URL: http://payment-service:8000
11    SHIPPING_SERVICE_URL: http://shipping-service:8000
12  depends_on:
13    mysql-order:
14      condition: service_healthy # Cho doi DB san sang
15    payment-service:
16      condition: service_started
17  networks:
18    - microservices-network

```

Lưu ý thiết kế: order-service khai báo `depends_on` cả `mysql-order` (health check) lẫn `payment-service` và `shipping-service` (started) vì order cần gọi sang hai service này trong luồng thanh toán.

4.5.3 Dockerfile (Django service)

Mỗi Django service dùng Dockerfile chuẩn:

```

1 FROM python:3.10-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 8000
7 CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]

```

4.5.4 Cấu hình Nginx Gateway thực tế

File nginx.conf định nghĩa upstream và routing cho tất cả services:

```

1 upstream product_service { server product-service:8000; }
2 upstream recommender_ai_service {
3     server recommender-ai-service:8000;
4 }
5
6 server {
7     listen 80;
8     # Routing API den tung service
9     location /api/v1/products/ {
10         proxy_pass http://product_service;
11     }
12     location /api/v1/chatbot/ {
13         proxy_pass http://recommender_ai_service;
14     }
15     # Tat ca request khac -> Django frontend
16     location / {
17         proxy_pass http://frontend;
18         proxy_set_header Host $host;
19         proxy_set_header X-Real-IP $remote_addr;
20     }
21 }

```

4.5.5 Khởi chạy hệ thống

```

1 # 1. Build tat ca image
2 docker-compose build
3
4 # 2. Khoi chay nen (detached)
5 docker-compose up -d
6
7 # 3. Khoi tao du lieu mau
8 ./setup_db.bat
9 # -> Tao 10 categories, 30 san pham, 14 reviews

```

```

10
11 # 4. Kiểm tra trạng thái
12 docker-compose ps
13 docker-compose logs -f order-service

```

4.6 Authentication & Security

4.6.1 JWT Authentication

Hệ thống sử dụng JWT (JSON Web Token) qua thư viện `djangorestframework-simplejwt`:

```

1 # Cài đặt
2 pip install djangorestframework-simplejwt
3
4 # settings.py - auth-service
5 REST_FRAMEWORK = {
6     'DEFAULT_AUTHENTICATION_CLASSES': [
7         'rest_framework_simplejwt.authentication',
8         '.JWTAuthentication',
9     ]
10 }
11
12 # urls.py - auth-service
13 from rest_framework_simplejwt.views import (
14     TokenObtainPairView, TokenRefreshView)
15 urlpatterns = [
16     path('auth/login/', TokenObtainPairView.as_view()),
17     path('auth/refresh/', TokenRefreshView.as_view()),
18 ]

```

Luồng xác thực:

1. User gửi username/password → auth-service trả về `access_token` và `refresh_token`.
2. Client lưu token, gửi kèm trong header: `Authorization: Bearer <token>`.
3. Mỗi API verify token qua middleware, từ chối request không hợp lệ (HTTP 401).
4. Khi token hết hạn, dùng `refresh_token` để lấy token mới không cần đăng nhập lại.

4.6.2 RBAC (Role-Based Access Control)

Phân quyền được thực thi ở cả API Gateway và từng service:

- **Admin/Manager:** Toàn quyền CRUD sản phẩm, xem tất cả đơn hàng, quản lý tài khoản.
- **Staff:** Xem và cập nhật trạng thái đơn hàng, xử lý vận chuyển.

- **Customer:** Mua hàng, xem lịch sử đơn, đánh giá sản phẩm đã mua.

4.6.3 Membership & Discount System

Hệ thống có cơ chế thành viên tự động tính discount khi checkout:

- **Bronze:** 0% discount (mặc định khi đăng ký)
- **Silver:** 5% discount
- **Gold:** 10% discount
- **Platinum:** 15% discount

Discount được tính tự động tại api-gateway bằng cách lấy hạng thành viên từ customer-service trước khi tạo order.

4.7 Monitoring Logging

4.7.1 Health Checks

```
1 GET http://localhost/health/
```

4.7.2 Logs

- Docker logs cho mỗi container
- Application logs trong Django

4.8 Performance Considerations

4.8.1 Scalability

- Scale từng service độc lập
- Ví dụ: scale product-service nếu nhiều traffic

4.8.2 Caching

- Product data có thể cache ở gateway
- Rating summary cache ngắn hạn

4.8.3 Database Indexing

- Index trên `productidtrongreviews`
- Index trên `categoryidtrongproducts`

4.9 Troubleshooting

4.9.1 Reviews không hiển thị

- Kiểm tra `comment-rate-service` đang chạy
- Verify review data được POST thành công
- Kiểm tra field: `productid, customerid`

4.9.2 AI Chat lỗi kết nối

- Kiểm tra `recommender-ai-service` running
- Verify CORS headers: `Access-Control-Allow-Origin`
- Check logs: `docker-compose logs recommender-ai-service`
- Endpoint phải là `/api/ai/chat/` (không `/chatbot/`)

4.9.3 Hình ảnh sản phẩm không hiển thị

- Verify `imageurlfieldpopulatedtUnsplash`
- Check network connectivity tới Unsplash CDN
- Verify image URLs escaped properly (*thaytrongbatch*)

4.10 Bài tập thực hành sinh viên

4.10.1 Mục tiêu

Sinh viên cần:

- Triển khai toàn bộ microservices
- Test từng API endpoint
- Verify reviews hiển thị trên products

- Test AI chatbot hoạt động
- Chạy full flow mua hàng

4.10.2 Step-by-step

1. Clone repo
2. Build: docker-compose build
3. Start: docker-compose up -d
4. Setup data: `./db.bat`
4. Test: `http://localhost`
5. Verify reviews: Check product pages
6. Test AI: Click robot, gửi tin nhắn

4.10.3 Verification Checklist

- 10 categories được tạo
- 30 products hiển thị với hình ảnh
- 14 reviews visible trên products
- Ratings display (4-5 sao)
- AI chatbot response không lỗi
- Cart functionality hoạt động
- Order flow hoàn thành

4.11 Kết luận

- Hệ thống microservices hoàn chỉnh với 11 services
- Data sample: 30 products, 10 categories, 14 reviews
- AI integration cho personalized recommendations
- Docker deployment sẵn sàng production
- Fully functional e-commerce platform

Kết luận

Tiểu luận này đã trình bày toàn diện quá trình xây dựng một hệ thống E-Commerce hiện đại theo kiến trúc Microservices, từ lý thuyết nền tảng đến triển khai thực tế.

Những gì đã đạt được

Về kiến trúc: Hệ thống được phân rã thành 11 microservices độc lập theo đúng nguyên tắc DDD (Domain Driven Design). Mỗi bounded context (User, Product, Cart, Order, Payment, Shipping, Review, AI) tương ứng với một service riêng biệt, có database riêng, deploy độc lập — đúng theo tiêu chí database-per-service.

Về công nghệ: Dự án áp dụng đúng hai loại database theo yêu cầu môn học: MySQL cho các service giao dịch (cart, order, payment, shipping) và PostgreSQL cho product-service (cần JSON và quan hệ phức tạp). Neo4j được dùng cho Knowledge Graph của AI service.

Về AI Service: AI service triển khai đầy đủ pipeline Hybrid Model:

- **LSTM:** Mô hình sequence modeling dự đoán sản phẩm tiếp theo dựa trên chuỗi hành vi người dùng.
- **Knowledge Graph (Neo4j):** Lưu quan hệ User–Product (BUY, VIEW, SIMILAR) để gợi ý theo network.
- **RAG + LLM:** Chatbot tư vấn kết hợp Retrieval-Augmented Generation với Google Gemini, trả lời bằng tiếng Việt.

Về triển khai: Toàn bộ hệ thống được Docker hóa và khởi chạy bằng một lệnh duy nhất (docker-compose up), đáp ứng tiêu chí “Docker chạy được” trong checklist đánh giá.

Bài học kinh nghiệm

- **Microservices không phải lúc nào cũng đơn giản hơn:** Complexity tăng đáng kể so với monolithic, đặc biệt trong quản lý distributed transactions (khi order-service phải phối hợp payment và shipping).

- **API Gateway là trung tâm:** Nginx đóng vai trò quan trọng, việc cấu hình sai routing là nguồn gốc của nhiều lỗi (ví dụ: lỗi HTTP 403 CSRF trên chatbot endpoint).
- **Database-per-service tốn tài nguyên:** Mỗi service có DB container riêng, tổng cộng 9 MySQL instance + 1 PostgreSQL + 1 Neo4j — phù hợp học tập nhưng cần tối ưu cho production.
- **DDD giúp phân chia trách nhiệm rõ ràng:** Việc xác định Bounded Context trước khi code giúp tránh được tình trạng service biết quá nhiều về nhau.

Hướng phát triển

- Tích hợp Kubernetes cho orchestration production.
- Thêm ELK Stack (Elasticsearch, Logstash, Kibana) cho monitoring tập trung.
- Implement message queue (RabbitMQ/Kafka) cho asynchronous events.
- Cải thiện LSTM model với dữ liệu thực tế và fine-tuning.